

文献引用格式: 和达, 陶锐, 詹国梁, 等. 智能体驾驭工程: 分析 Claude Code 技术实现 [J]. 通信技术, 2026, 59(7): 779–787.

DOI:10.3969/j.issn.1002-0802.2026.07.007

智能体驾驭工程: 分析 Claude Code 技术实现^{*}

和 达^{1,2}, 陶 锐^{1,2}, 詹国梁^{1,2}, 何秉钧^{1,2}, 王炳文^{1,2}, 毛得明^{1,2}

(1. 中国电子科技集团公司第三十研究所, 四川 成都 610041;

2. 中国电子科技网络信息安全有限公司, 四川 成都 610041)

摘 要: “人工智能+”是国家级战略, 将驱动各行各业积极布局智能体。2026 年 3 月 Claude Code 的源码泄露, 为业界提供了首个工业级编程智能体的完整架构视图。首先阐明驾驭工程的定义, 厘清其与传统智能体框架的区别。其次以泄露的源码为样本, 审视其背后的驾驭工程设计, 重点剖析五大核心机制, 包括信息熵治理、流程确定性管控、智能体回环等, 主张企业应着重聚焦驾驭工程的实现而非大模型本身的能力提升, 可将视野聚焦在智能体群体智能这一领域。本文为智能体相关从业人员的后续研究提供了参考与实践指引。

关键词: 智能体; 驾驭工程; Claude Code; 大模型

中图分类号: TN91; TP311 **文献标志码:** A **文章编号:** 1002-0802(2026)-07-0779-09

Agent Harness: An Analysis of the Implementation of Claude Code

HE Da^{1,2}, TAO Rui^{1,2}, ZHAN Guoliang^{1,2}, HE Bingjun^{1,2}, WANG Bingwen^{1,2}, MAO Deming^{1,2}

(1.No.30 Institute of CETC, Chengdu Sichuan 610041, China;

2.China Electronics Technology Cyber Security Co., Ltd., Chengdu Sichuan 610041, China)

Abstract: The “AI+” initiative has been elevated to a national strategy and is poised to drive proactive deployment of AI agents across industries. The source code of Claude Code was leaked in March 2026, offering the first comprehensive view of the architecture of an industrial-grade AI agent. This paper begins by clarifying the definition of harness engineering and distinguishing it from conventional agent frameworks. It then uses the leaked source code as a case study to examine the underlying harness engineering design. The analysis focuses on five core mechanisms, including information entropy governance, deterministic process control, and agent loops. The paper argues that enterprises should prioritize the implementation of harness engineering rather than solely enhance the capabilities of large language models, and suggests shifting focus toward the domain of swarm intelligence in multi-agent systems. This paper provides both theoretical reference and practical guidance for researchers and practitioners working with intelligent agents.

Keywords: agent; harness engineering; Claude Code; large language model

0 引 言

2026 年 3 月 31 日, 安全研究员 Chaofan Shou

在 npm 注册表中监测发现, Anthropic 在发布 Claude Code v2.1.88 版本中时, 不慎附带了一份 59.8 MB 的

^{*} 收稿日期: 2026-05-07; 修回日期: 2026-06-12 Received date: 2026-05-07; Revised date: 2026-06-12

基金项目: 国家自然科学基金联合基金重点支持项目 (U24B20147)

Foundation Item: Joint Funds of the National Natural Science Foundation of China (U24B20147)

source map 文件。该文件可将压缩打包代码完整还原为原始 TypeScript 源码,致使 1 900 余个源文件、总计 51.2 万行核心代码被公开到公共互联网上。事件发生数小时内,相关备份仓库在 GitHub 上获得上万星标量,同时冲上 Hacker News、Reddit 等主流技术社区热榜。

此次源码泄露事件在学术界和工业界引发强烈反响,其原因在于:随着智能体从基础对话交互演进到自主执行复杂任务,决定智能体成败的因素已不再局限于大模型本身的“智力”,而是包裹在大模型周围的工程化架构体系。该体系依托大模型基础调用能力,构建起包含多步逻辑推理、工具使用、记忆管理、任务委派和停止机制的控制栈,以及检索增强生成(retrieval-augmented generation, RAG)和显式自我反馈。针对该体系的研究工作近期已扩展到长任务周期上下文管理、记忆自进化、智能 workflow 构建、多智能体协同编排和原生工具高效执行等方向。

2026 年初,业界将此类工程化外壳统称为驾驭工程(harness engineering)^[1],其核心理念可总结为“人在环路上掌舵,智在环路中操控”。Claude Code 作为行业头部编程类智能体,其源码的外泄,恰好为学界与业界提供了极具研究价值的驾驭工程落地案例。

本文将研究以下 3 大核心问题:

(1) Claude Code 的 Harness 架构的设计逻辑与层级架构;

(2) 该架构体系背后的核心设计理念;

(3) 依托本次源码案例,提炼出能够通用至其他智能体系统的标准化设计准则。

本文以泄露源码的公开分析报告为主要研究资料,系统梳理 Claude Code 的相关技术实现方案。

1 相关工作

本节依次梳理智能体发展脉络,剖析提示词与上下文工程的发展成效及现存短板,并阐述驾驭工程的研究概况。

1.1 智能体系统的研究现状

要理解智能体,首先要厘清近年来大模型系统的演进脉络^[2]。

(1) 聊天机器人阶段。该阶段的大模型系统属于无状态的问答工具,每次对话均相互独立,记忆留存和任务持续执行能力较弱。此时的大模型仅能根据当前提示词生成响应,不具备工具调用和行

动能力。

(2) 工作流阶段。以 n8n、LangChain 等工具为代表,该阶段的大模型系统嵌入由代码驱动的有向无环图(directed acyclic graph, DAG),其每一步操作均由预定义 workflow 决定。

(3) 自主智能体(autonomous agent)阶段。该阶段大模型自主控制执行循环,系统不再预设每一步的走向,而是赋予大模型充分的自主决策权限。Claude Code 正是第 3 代智能体的商业化典型代表。

大模型与智能体之间的界限到底是什么?作者认为,大模型是具备多模态理解和多模态生成能力的人工智能模型;而智能体是具备环境感知和环境影响能力的系统,其概念早于大模型。早在 1995 年, Russell^[3] 就在其经典教材中指出,凡能够通过传感器感知环境并借助执行器作用于环境的实体,均可视为智能体。只是在当前语境下,智能体更多指代基于大模型构建的智能体系统。

智能体聚焦于构建具备感知、决策、执行、反馈能力的智能系统。在驾驭工程之前,其框架为

$$\text{Agent} = \text{LLM} + \text{Plan} + \text{Tool} + \text{Memory}$$

研究方向集中在任务规划、工具调用、记忆管理模块。现有研究已经验证了智能体在代码生成、数据分析、自动化办公等场景的应用潜力。2026 年初,图标设计为龙虾的 OpenClaw 掀起“养龙虾”热潮^[4],进一步将人工智能从对话工具向生产力引擎演进,阿里、华为、腾讯亦相继落子智能体领域研发。

目前,除了少数商业公司推出的编程类智能体工具已实现落地,多数智能体仍停留在原型验证阶段,难以真正应用于真实生产环境。当前智能体研究存在 3 大核心缺口。

(1) 多数研究聚焦于算法层面的创新,忽视了工程化体系对智能体性能的决定性作用,导致出现“实验室效果好,实际应用效果不佳”的问题;

(2) 现有智能体架构普遍缺乏对大模型概率性输出的系统性约束,无法保证智能体行为的确定性与可预测性;

(3) 缺乏工业级、全链路的实现案例作为参考,多数开源智能体仅实现了核心循环,缺失完整的安全管控、资源优化、可观测性等生产级必备能力。

1.2 提示词工程、上下文工程的发展与局限

提示词工程(prompt engineering)是智能体的初代范式^[5],其核心目标是通过优化输入提示词的结构、内容与格式,引导大模型输出符合预期的结果,核心技术包括零样本提示、少样本提示、思维

链 (chain of thought, CoT) 提示等。在单轮交互、低复杂度场景中, 提示词工程能显著提升大模型输出的准确性, 成为 2023—2024 年智能体应用的核心技术手段。

2025 年, 智能体的研究重点转向上下文工程 (context engineering)。与提示词工程不同, 上下文工程将研究焦点从“给大模型输入什么提示词”扩展到大模型当前能获取的全部动态信息^[6], 包括检索内容、对话历史、工具上下文、记忆信息、状态摘要等, 旨在让大模型的每一步决策与响应都获得恰当、简洁、精准的上下文输入, 进而提升输出的合理性与连贯性。

然而, 提示工程和上下文工程均存在明显局限, 既未覆盖权限管控、执行验证、运行时策略、故障恢复、长链任务执行等关键能力, 也无法突破大模型的固有短板, 具体表现如下。

(1) 鲁棒性不足, 提示词的优化效果高度依赖大模型版本, 模型升级后, 原有优化策略往往直接失效;

(2) 复杂任务处理能力弱, 随着交互轮次增加, 大模型极易出现上下文混淆、任务偏航等问题, 难以应对长时程、多步骤的复杂任务;

(3) 错误处理能力欠缺, 只能被动依赖人工修正, 无法实现自主闭环纠错;

(4) 安全管控完全依赖大模型自身的对齐效果, 无法构建系统级安全护栏。

对于需自主运行、高可靠性的智能体系统而言, 仅依靠提示词工程或上下文工程无法满足生产级场景的核心需求。

1.3 驾驱工程的兴起

将大模型比作一匹“烈马”, 驾驱工程即为“烈马”套上“缰绳”与“马鞍”, 其目的是正确地引导大模型, 提供合理的运行环境与约束机制, 实现具体业务场景的长链任务自主执行。该概念由 Hashimoto 于 2026 年 2 月正式提出^[1], 是当前智能体领域兴起的全新模式。可将驾驱工程定义为: 围绕大模型构建的, 用于实现智能体行为确定性、任务可靠性、安全可控性与长链任务执行能力的全链路工程化体系, 是智能体的“操作系统”。其核心思想是: 不再试图改变大模型本身, 而是为大模型构建一套完整的运行环境与基础设施体系, 通过约束规则、反馈闭环、验证标准、安全护栏等工程化手段, 确保智能体在无人干预的情况下, 仍能自主、

安全、可靠地完成任务。相应地, 基于驾驱工程的智能体框架也升级为

Agent=LLM+Harness

这种为大模型“加壳”的工程化管控方式, 其有效性已被证实。早在 2024 年, Jimenez 等人^[7]在使用相同基座模型的前提下, 仅优化执行环境、交互接口、反馈回路与错误恢复机制等管控层设计, 使智能体对 SWE bench 任务的解决率从 12% 大幅提升至 76%, 直观说明相同大模型在不同管控机制下可呈现量级差异的表现^[8]。Bölük^[9]通过一项直观的对比实验证实, 在完全不改变大模型权重、提示词与训练数据的前提下, 仅通过优化大模型外部的驾驱层设计, 包括执行接口、代码反馈回路、工具调用规则与错误修复机制, 就能够让 15 种不同的大模型在代码任务上获得一致且显著的性能提升。He 等人^[10]、Pan 等人^[11]相继提出智能体驾驱工程的框架, 进一步丰富了驾驱工程的理论与实践体系。

2 Claude Code 的驾驱工程

从泄露的源码可知, Claude Code 采用 TypeScript 开发, 源代码共包含约 1 900 个源文件, 代码总量超过 51.2 万行, 集成各类工具 40 余种。系统核心组件包括大型查询引擎、集中式工具注册表、数十类斜杠命令、持久化记忆系统、集成开发环境 (integrated development environment, IDE) 桥接、工具调用 (model context protocol, MCP) 集成、远程会话、插件系统、技能系统 (Skills) 等。该智能体包含 6 层架构, 分别是用户界面层、编排层、智能层、能力层、安全治理层和基础设施层, 分层架构实现了业务关注点分离, 各组件依托标准化接口完成交互。

如图 1 所示, 不同于开源社区的原型项目, 该源码是经过大规模用户验证的商业系统, 涵盖了从 UI 实现、核心循环到边缘管控的全链路设计, 本文仅围绕其中与驾驱工程相关的组件进行针对性分析。

2.1 提示词: 动态加载机制

Claude Code 的系统提示词并非硬编码字符串, 而是从 6 个不同来源实时组装生成的, 如表 1 所示。这种分层组装的优势在于, 每层都可以独立更新, 互不干扰。例如, 当用户通过 MCP 接入新工具时, 只需要更新工具层, 不影响系统层和项目层的内容。再比如, 智能体从普通模式切换到协作模式, 只需要替换模式层的提示词, 其他 5 层保持不变。

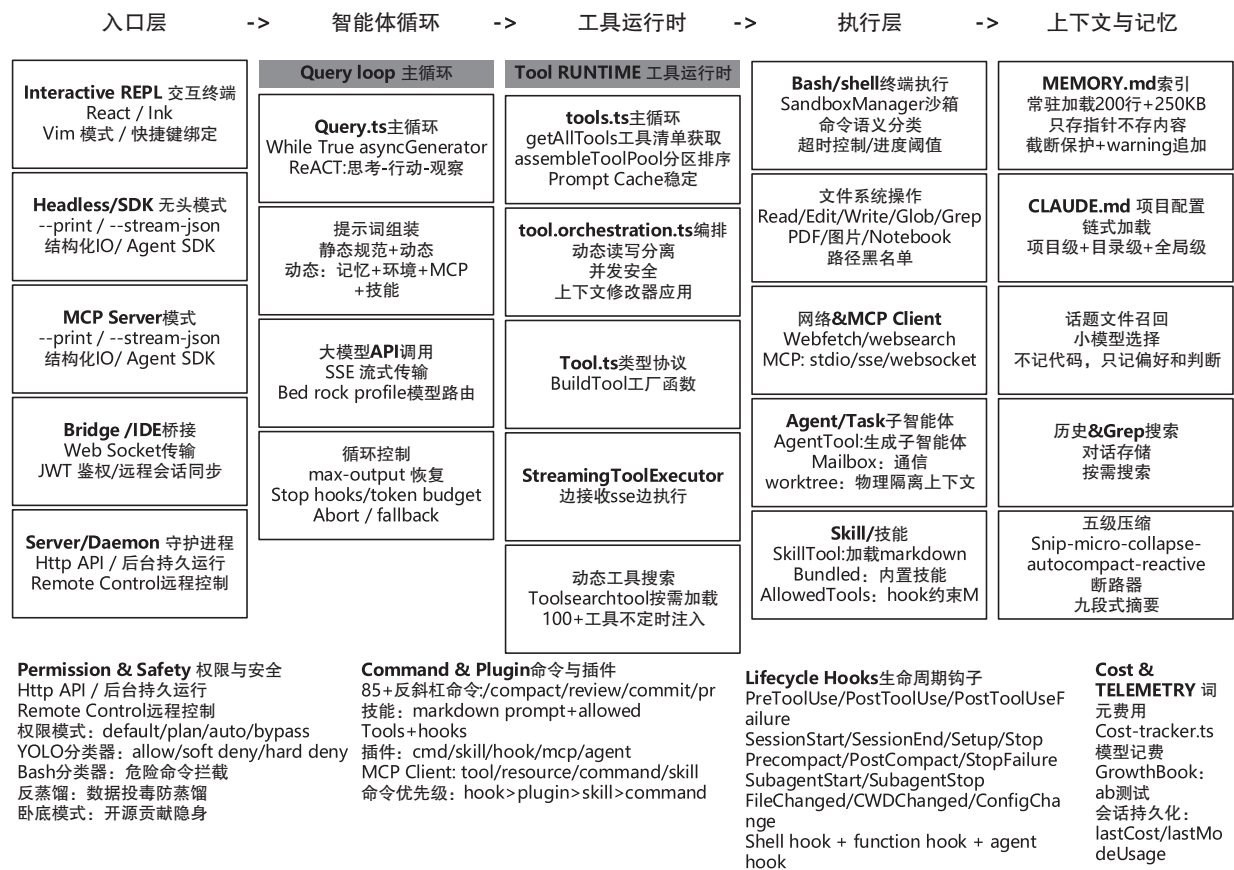


图 1 Claude Code 架构^[12]

这种动态加载机制的背后，隐含着贯穿 Claude Code 工具加载、技能加载、提示词加载、代码加载、记忆加载的核心逻辑——渐进式披露（progressive disclosure）。其核心内涵为：信息并非一次性输入，而是按需获取，分层展开，经历从“单点线索”到“完整主体”再到“深层解析”的逐层暴露、逐步解析过程。提示词层级如表 1 所示。

表 1 提示词层级

层 次	来 源	内 容	更新频率
系统（System）	硬编码	基础角色定义、安全规则	每版本更新
模式（Mode）	模式选择	Coordinator/Worker/Normal 行为规范	每次模式切换
工具（Tools）	已注册工具	工具描述和使用规范	MCP 连接时
项目（Project）	Claude.md	项目级持久记忆	用户 / 智能体主动更新
版本（Git）	Context.js 实时读取	仓库状态快照（分支、改动文件）	每轮对话
历史（History）	记忆 + 压缩摘要	会话上下文	每轮对话

2.2 信息熵治理：记忆层级架构与压缩机制

具备长链任务执行能力的智能体，普遍面临“上下文熵增”的挑战，即随着会话轮次增加，对话历史、工具执行结果等信息不断累积，进而导致上下文窗口溢出、大模型注意力分散、幻觉频发等问题。对此，Claude Code 设计了层级记忆架构与上下文压缩机制，构建了完整的信息层驾驭体系，有效解决了智能体长链任务执行过程中的“失忆”难题。

与常见智能体“全量历史对话加载”的设计思路不同，Claude Code 采用了分层记忆架构，实现了

记忆的按需加载与精准检索。具体分为 3 级记忆层次，各层次功能与特性明确区分。

1 级记忆（常驻层）：对应 memory.md 轻量索引层，仅存储指向实际数据的指针与任务核心元信息，包括任务目标、关键节点、核心文件路径等。该层始终加载在上下文中，占用词元量极少，相当于智能体的“工作目录”；其大小限制在 200 行以内，且每行不超 150 个字符，若超出限制将触发行数截断和字节截断机制。

2 级记忆（按需层）：对应 Topic Files 主题文件层，

将项目知识、领域规则、历史执行详情等分布式存储在各个主题文件中。智能体仅在需要相关信息时, 通过检索工具实时抓取对应内容, 无须全量加载至上下文, 相当于智能体的“知识库”。

3 级记忆 (冷备层): 对应原始对话历史层。完整的对话记录与工具执行结果从不全量加载回上下文, 而是归档保留。当需要溯源时, 仅通过 grep 式检索方式, 针对特定标识符进行快速搜索, 提取对应片段该层相当于智能体的“归档库”。

这套分层记忆架构的核心驾逻辑在于, 通过

“索引—主题—归档”的 3 级分层, 让智能体在长时程运行中, 始终只将最核心的信息加载到上下文窗口中, 既保证了任务目标不偏离, 又避免了无效信息对大模型的干扰, 从根本上解决了上下文熵增问题。

针对工具执行结果、对话历史等动态增长的内容, Claude Code 设计了 5 级渐进式上下文压缩机制。该机制可根据词元预算与内容重要性, 自动选择对应的压缩级别, 在保证信息完整性的前提下, 最大化节省词元消耗。5 级压缩机制具体如表 2 所示。

表 2 5 级压缩机制

压缩级别	核心逻辑	适用场景
裁剪 (Snip)	仅保留工具调用的结构与执行状态, 不保留具体内容	执行成功的无返回值操作、重复执行的无变化操作
微压缩 (Micro Compact)	大体积工具结果的摘要, 仅保留核心结论	超长日志、大文件读取结果、批量操作执行结果
上下文折叠 (Context Collapse)	中间对话的折叠处理, 仅保留关键决策与核心结论, 去除冗余思考过程	多轮迭代的中间步骤、已验证的执行过程
自动压缩 (Auto Compact)	当上下文占用超阈值时, 自动对非核心内容进行摘要压缩	长时程会话、词元占用接近预算上限的场景
响应式压缩 (Reactive Compact)	当返回“上下文溢出”错误时, 触发紧急全量压缩, 保证循环不中断	极端长会话、超大体积内容处理场景

通过这套 5 级压缩机制, Claude Code 实现了上下文占用的动态管控。实测数据表明, 在 20 轮以上的长时程开发任务中, 该机制可节省 60% 以上的词元消耗, 同时任务完成率相比原来提升 4 倍以上, 较好地解决了传统智能体“会话越长, 效果越差”的问题。

2.3 流程确定性管控: 钩子机制

为实现对工具执行全流程的确定性管控, Claude Code 设计了完整的生命周期钩子机制, 如图 2 所示。

钩子是软件工程领域广泛应用的模块化扩展与流程管控手段, 即在软件系统的标准执行流程中预设的、可被外部逻辑挂载与拦截的执行节点, 能够在不修改核心源码的前提下, 实现流程干预、功能扩展、权限校验与运行状态监控。从技术本质上看, 钩子融合了面向切面编程 (aspect-oriented programming, AOP) 与事件驱动的设计模式, 可实现系统的核心业务逻辑与扩展控制逻辑完全解耦, 目前已广泛应用于操作系统、中间件、Web 框架、IDE 插件体系等工程实践。

这一经典模式, 正是 Claude Code 驾工程中流程管控体系的理论源头与工程基础, 也是 Claude Code 驾工程的核心创新之一。其价值在于: 将所

有确定性的规则与约束逻辑, 从大模型的概率性推理中剥离出来, 通过代码实现了大模型的确定性执行。无论大模型输出内容如何变化, 系统都能在关键节点触发预设管控逻辑, 从根本上杜绝了大模型的“不可预测行为”。Claude Code 主要的钩子类型如表 3 所示。



图 2 驾工程钩子机制

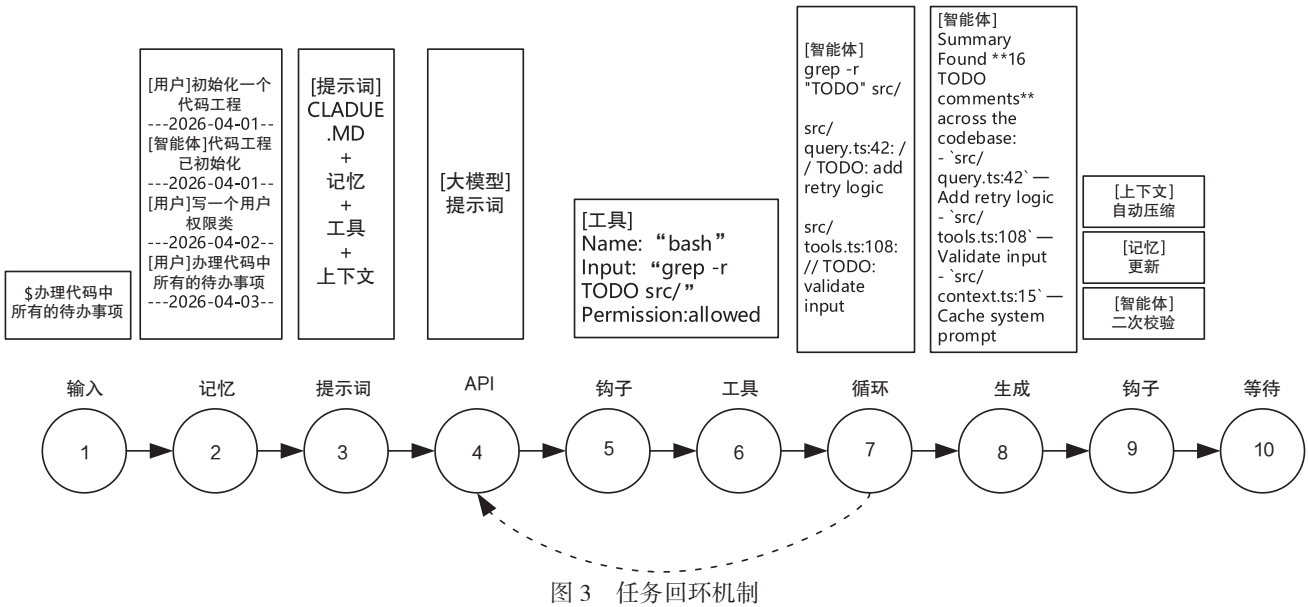
表 3 Claude Code 主要的钩子类型

钩子节点	执行时机	用 途
提示词钩子	用户输入提交后，发送给大模型前	输入预处理、格式校验、安全过滤、任务拆解
前置钩子	大模型生成工具调用后，执行前	工具权限校验、风险评估、参数校验、指令拦截
后置钩子	工具执行完成后，结果返回给大模型前	结果压缩、敏感信息过滤、格式标准化、审计日志记录
环路状态钩子	单次循环结束后，下一轮循环开始前	上下文压缩、任务进度评估、死循环检测、预算管控
任务状态钩子	任务结束后，结果返回给用户前	结果校验、自动测试、文档生成、状态清理

2.4 任务执行模式：回环机制

与多数开源智能体采用的多智能体协同、并行架构不同，Claude Code 采用了单线程循环的极简设计。智能体任务执行经历“思考—行动—观察—重复”的环路，如图 3 所示，该设计与 LangChain 的路线形成了鲜明对比：LangChain 倾向于用复杂的编排控制智能体，而 Claude Code 则将推理、决策

和停止判断完全下放给大模型。值得注意的是，该任务执行环路的核心逻辑只有约 50 行代码，却能为大模型赋予几乎无限的操作空间。这种极简设计实际上蕴含着更为深刻的见解：随着大模型能力的提升，复杂的编排只会成为大模型的累赘，合理的架构设计应是 将智能全部交给大模型，将确定性留给框架。



2.5 工具系统与权限管控机制

工具系统是智能体对接现实世界的交互接口，也是驾驭工程实现行为约束的核心载体。Claude Code 的工具系统采用插件化架构，内置 40 余种独立工具模块，每个工具都实现了严格的风险分级与权限管控，形成了完整的执行层驾驭体系。

在工具选择方面，Claude Code 同样打破了常规工程实践。其高度依赖 Bash 类工具完成任务，如在智能体开展内容检索时，Claude Code 未采用行业主流的 RAG 知识库构建方案，而是将决策权交给大模型，通过 Glob 工具和 Grep 工具关键字匹配的方式自主获得上下文，规避了传统 RAG 方案因原始数据分片、特征抽取、检索算法导致的信息损失问题。

在工具的权限管控方面，Claude Code 根据风险等级将所有工具划分为 3 级，并设置了差异化管控策略。

(1) 低风险工具：包括文件读取、代码搜索、信息查看等只读操作，无须用户确认即可直接执行，同时设置了单次读取文件大小上限、并发读取数量限制，防止资源耗尽。

(2) 中风险工具：包括文件编辑、代码格式化、Git 提交等修改操作，执行前会向用户展示 diff 预览，同时设置自动备份机制，操作失败时可自动回滚。

(3) 高风险工具：包括 Bash 命令执行、系统调用、网络请求等可能影响系统安全的操作，执行前必须经过风险关键词拦截、用户二次确认、沙箱环境隔离三重校验。

同时, 系统内置了命令黑名单, 拦截 `rm-rf`、格式磁盘等高危指令, 每个工具在注册时需要声明其所需的权限级别。此外, Claude Code 还搭配使用 YOLO 分类器, 判断操作的危险程度, 作为一种拦截预期之外的高风险操作的兜底。

其余的词元管理、反蒸馏与卧底、虚拟宠物 Buddy 等模式与驾驭工程相关性较小, 不在此赘述。通过上述分析, Claude Code 各类功能模块都围绕“约束大模型不确定性, 提升系统可靠性”进行设计。

3 大模型未来发展

当前, 全球智能产业正经历从“模型能力竞赛”向“工程化落地”的关键转折。我国“十五五”规划纲要将“深化拓展‘人工智能+’”“全面实施‘人工智能+’行动”^[13]纳入顶层战略布局。人工智能产业发展已上升为大势所趋的国家战略。通过审视 Claude Code 智能体背后的驾驭工程设计, 本文从智能体生态格局、多智能体组织形态及落地实施策略 3 个维度进一步展望未来发展。

3.1 big model 与 big harness 之争

关于智能体的发展路径, 有两个旗帜鲜明的阵营, 即在构建垂直领域智能体时, 是该将有限的资源投入给大模型 (big model) 还是设计更厚的“外壳” (big harness) ?

大模型派主张, 模型能力提升是解决智能体任务规划错误、工具调用错误、上下文理解偏差等问题的根本途径, 当前智能体落地难是由于大模型不够“聪明”。

“外壳”派则认为, 大模型能力的提升是有上限且边际收益递减的, 而真实世界任务的复杂性和多样性是无界的, 因此工程化外壳的质量将最终决定智能体的上限。

本文持融合辩证观点: big model 和 big harness 实际上是硬币的两面, 大模型的“智商”决定了智能体能力的上限, 而驾驭工程的“素养”决定了智能体在真实环境中能够发挥出的水平。big model 是智能体发展的最终抉择, 而 big harness 是当下智能体落地的优选抉择。

3.2 从集群到群智: agent swarm 的本质思考

agent swarm 直译过来就是智能体“集群”, 但 swarm 还有另外一种翻译——“蜂群” (实际是指群体智能)。集群的特征是任务空间分解与并行执

行。在这种模式下, 智能体之间没有实质性的交互与协商, 协同关系是预设的、静态的。蜂群则有着完全不同的内涵, 它源自生物界, 其核心在于信息素的交换涌现智能, 没有中央指挥, 个体遵循简单的局部规则, 通过相互感知和反馈, 整体呈现出超越个体线性叠加的复杂智能行为^[14]。从 Claude 的最新发布的 Agent Team^[15]中, 可清晰窥见其在群体智能方向的布局。其官方文档中明确表示, 智能体间可以相互通信, 可以互相挑战观点, 共享任务列表, 具备自协调能力。从泄露的 Claude Code 代码和核心开发者 Tarih 的博客^[16]分析可知, 其驾驭工程的底层逻辑是让智能自行涌现, 而非预先编排, 这都与集群的概念背道而驰。放眼全球智能体产业生态, Anthropic 凭借 Claude 系列大模型及智能体产品, 奠定了世界领先的地位, 同时通过制定 MCP 和 Skills 等机制锁定了智能体的基建设施, 该企业已处于生态位头部。智能体“群智”或将成为下一个战略方向。

3.3 智能体“脚手架”变薄还是变厚?

Claude Code 的设计哲学可以概括为“大道至简, 智能下移”, 这一理念本质上是对早期智能体框架重编排路线的反驳。这种设计基于一个明确预判: 大模型的核心能力在持续提升。那些试图在代码层预判模型行为、预设执行路径的“聪明”设计, 有 2 个根本性问题:

(1) 这类设计必然落后于大模型的能力演进, 因为框架开发者无法预见大模型未来可能发展出的能力。

(2) 这类设计限制了大模型的发挥空间——当大模型想要以框架设计者未曾设想的方式解决问题时, 会被“精细化”的设计所束缚。

因此, 智能体框架演进的方向应当是随着大模型能力的提升, 框架层应该变得越来越“薄”, 即“脚手架应随时间变薄”。如果每次大模型升级后, 都发现需要删除一部分硬编码的编排逻辑, 那么框架方向就是正确的; 反之, 如果在大模型升级后反而需要增加更多“护栏”和“引导”, 那么框架方向可能存在偏差。

这一命题与传统软件工程形成鲜明反差。在传统软件工程中, 随着系统演化, 开发者通常会增加更多代码来覆盖更多场景及处理更多边界情况。但在智能体领域, 大模型能力的提升, 使得原本需要通过代码来处理的问题, 可以直接交由大模型自

主处理。因此, 框架演化可能呈现一种“逆向增长”的轨迹——系统的代码量不增反减, 因为越来越多的逻辑可以从确定性代码迁移到大模型的推理中。

4 结 语

此次 Claude Code 的源码泄露, 虽属于意外工程事故, 却也是一次难得的行业知识溢出, 让从业者第一次完整窥见顶级智能体产品的工程化实现方案。它的架构揭示了一个被广泛承认但鲜少被系统阐述的真相: 在智能体时代, 决定成败的最大变量往往不是大模型有多聪明, 而是大模型被放在了一个什么样的环境里。

驾驭工程正是对这个“环境”的系统性工程化。通过对泄露代码的审视, 本文提炼了 3 点洞察。

(1) 对于绝大多数公司而言, 盲目投身 big model 并非最优路径, 相当于以己之力对抗行业头部的持续投入与进化, 更务实的选择是通过精巧的驾驭工程, 在现有大模型之上构建差异化的智能体能力。

(2) 当前多智能体架构停留在“主从式并行集群”层面, 缺乏真正的智能涌现。智能体“群智”可能是下一个战略方向。

(3) 智能体“脚手架”应随时间变薄, 随着大模型能力提升, 硬编码的补偿逻辑应主动删除, 框架变得更薄, 而非愈发臃肿。

综上, 驾驭工程理念及 Claude Code 泄露的代码实际上为从业者勾勒了一条更可靠、更可维护的垂直领域智能体落地的道路。结合国家“十五五”规划纲要, 智能体的发展已从技术探索上升为国家战略层面的必然选择。在此行业趋势下, 驾驭工程作为连接大模型能力与真实应用的工程桥梁, 其战略价值将进一步凸显。

参考文献:

- [1] Hashimoto M. My AI adoption journey[EB/OL]. (2026-02-05)[2026-04-01]. <https://mitchellh.com/writing/my-ai-adoption-journey>.
- [2] Xi Z H, Chen W X, Guo X, et al. The rise and potential of large language model based agents: A survey[PP/OL]. V3. arXiv(2023-09-19)[2026-04-02]. <https://arxiv.org/abs/2309.07864>.
- [3] Russell S J. Artificial intelligence: A modern approach[M]. New York: Pearson Education, 2010.
- [4] 孙亚慧. 国产 AI 应用从“会说话”到“会做事”[EB/OL]. (2026-03-25)[2026-04-02]. https://paper.people.com.cn/rmrhwb/pc/content/202603/25/content_30147227.html.
- [5] Schulhoff S, Llie M, Balepur N, et al. The prompt report: A systematic survey of prompt engineering techniques[PP/OL]. V6. arXiv(2025-02-26)[2026-04-02]. <https://arxiv.org/abs/2406.06608>.
- [6] Rajasekaran P, Dixon E, Ryan C, et al. Effective context engineering for AI agents[EB/OL]. (2025-09-29)[2026-04-02]. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- [7] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world GitHub issues?[PP/OL]. V3. arXiv (2024-11-11)[2026-04-02]. <https://arxiv.org/abs/2310.06770>.
- [8] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering[PP/OL]. V3. arXiv (2024-11-11)[2026-04-02]. <https://arxiv.org/abs/2405.15793>.
- [9] Bölük C. I improved 15 LLMs at coding in one afternoon, only the harness changed[EB/OL]. (2026-02-12)[2026-04-02]. <https://blog.can.ac/2026/02/12/the-harness-problem/>.
- [10] He C Y, Zhou X, Wang D, et al. Harness engineering for language agents: The harness layer as control, agency, and runtime [EB/OL]. (2026-03)[2026-04-02]. https://www.researchgate.net/publication/403120318_Harness_Engineering_for_Language_Agents_The_Harness_Layer_as_Control_Agency_and_Runtime.
- [11] Pan L Y, Zou L X, Guo S, et al. Natural-language agent harnesses[PP/OL]. arXiv (2026-03-26)[2026-04-02]. <https://doi.org/10.48550/arXiv.2603.25723>.
- [12] 程序员鱼皮. 刚刚 Claude Code 源码泄露! 我扒出了 11 个隐藏秘密 [EB/OL]. (2026-04-02)[2026-04-02]. <https://www.codefather.cn/post/2039551862717313025>.
- [13] 新华社, 中华人民共和国国民经济和社会发展第十五个五年规划纲要 [EB/OL]. (2026-03-13)[2026-04-02]. https://www.gov.cn/yaowen/liebiao/202603/content_7062633.htm.
- [14] Mitchell M. Complexity a guided tour[M]. Oxford: Oxford University Press, 2009.
- [15] Claude Code Docs. Orchestrate teams of Claude Code

sessions[EB/OL]. [2026-04-02]. <https://code.claude.com/docs/en/agent-teams>.

- [16] Thariq. Lessons from building Claude Code: Seeing like an agent[EB/OL]. [2026-04-02]. <https://x.com/trq212/status/2027463795355095314>.

作者简介:



和 达 (1992—), 男, 硕士, 高级工程师, 主要研究方向为软件安全;

陶 锐 (1997—), 男, 硕士, 工程师, 主要研究方向为软件安全;

詹国梁 (1997—), 男, 硕士, 工程师,

主要研究方向为软件安全;

何秉钧 (1996—), 男, 硕士, 工程师, 主要研究方向为软件安全;

王炳文 (1993—), 男, 硕士, 工程师, 主要研究方向为软件安全;

毛得明 (1982—), 男, 博士, 正高级工程师, 主要研究方向为网络安全、新型防御。